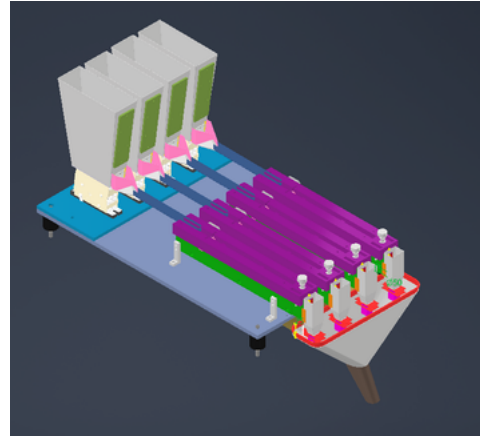


PHARMACEUTICAL MACHINE – AUTODESK INVENTOR



WHAT?

- Created detailed 3D [CAD](#) models of pharmaceutical machinery as part of my work at Lenis Machines.
- Modelled machine components and assemblies to support equipment design, layout, and future modifications.

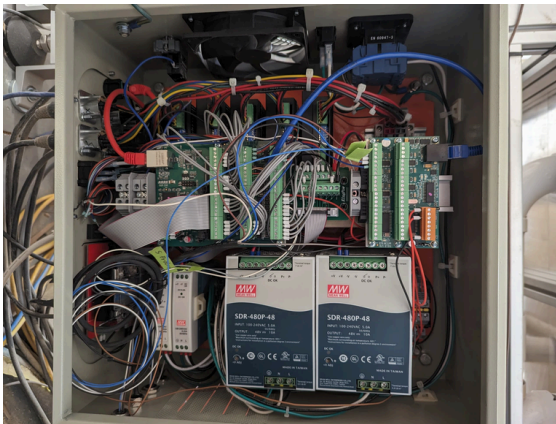
HOW?

- Used [Autodesk Inventor](#) to build parts and assemblies from existing machinery, measurements, and design requirements.
- Organized components into complete assemblies and checked positioning, fit, clearances, and mechanical interfaces.

RESULTS

- Produced accurate digital models that made the machine easier to visualize, document, and modify.
- Gained hands-on experience with 3D CAD, assembly design, mechanical systems, and industrial machine development.

LINUX-BASED INDUSTRIAL CONTROL SYSTEM



WHAT?

- Developed a [Linux-based](#) control system for the retrofit and modernization of an industrial CNC machine.
- Integrated PC-based control, [PLC](#) logic, industrial I/O, and machine hardware into a working automation platform.

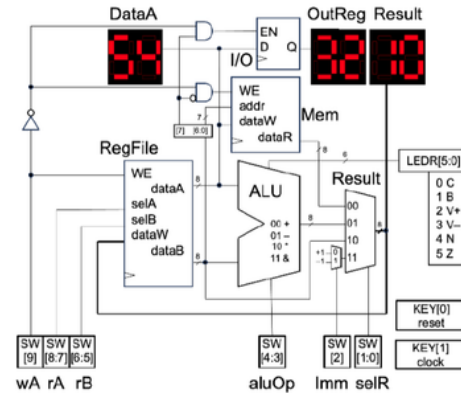
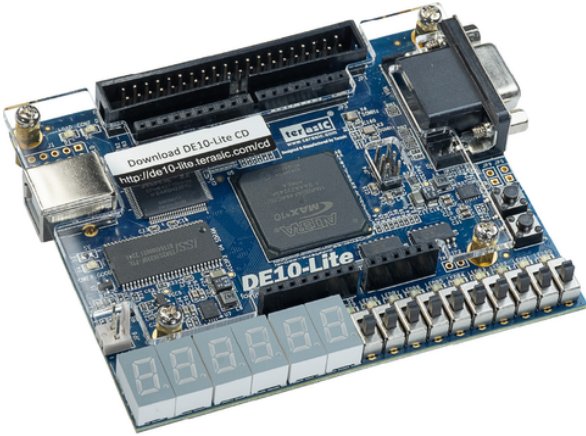
HOW?

- Configured Ubuntu, [LinuxCNC](#), [CODESYS](#), EtherCAT, and [Modbus](#) TCP on an industrial TinyPC.
- Integrated EtherCAT I/O, relays, sensors, and PLC logic while troubleshooting communication and hardware issues.

RESULTS

- Achieved reliable communication between CODESYS, LinuxCNC, and EtherCAT hardware.
- Successfully controlled physical relay outputs from software and established a working foundation for further machine automation.
- Gained practical experience in Linux, PLCs, industrial networking, commissioning, and systems integration.

SIMPLE CPU - SYSTEMVERILOG



WHAT?

- Designed and implemented a simplified CPU to execute a custom instruction set, focusing on datapath architecture, control logic, and instruction sequencing.

HOW?

- Built the CPU in SystemVerilog, developing the datapath (register file, ALU, buses) and a finite-state control unit.
- Verified functionality through simulation (Questa) and used the design through a DE10-Lite FPGA board for hardware testing and debugging.

RESULTS

- Successfully executed arithmetic, logic, and control instructions on real hardware, gaining hands-on experience with hardware-software interaction, cycle-level debugging, and translating CPU specifications into a working FPGA implementation.

ELECTRIC GO-KART- WELDING & WIRING



WHAT?

- Designed and built a functional go-kart from the ground up with two teammates using a custom metal chassis and drivetrain.
- Extended the project by converting the completed gas-powered vehicle to an electric drivetrain using an e-bike battery pack with integrated BMS.

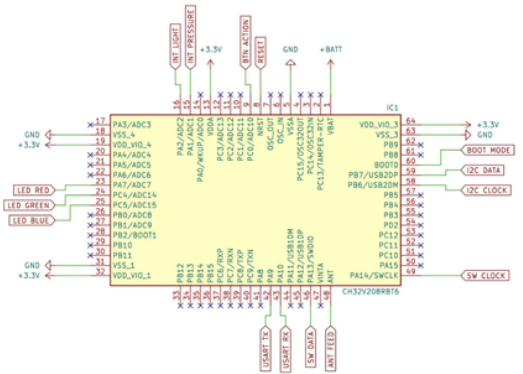
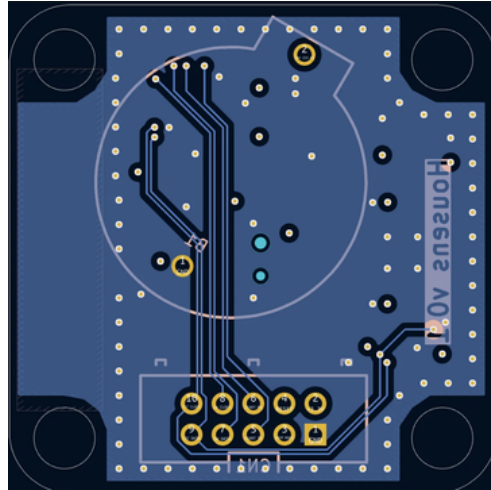
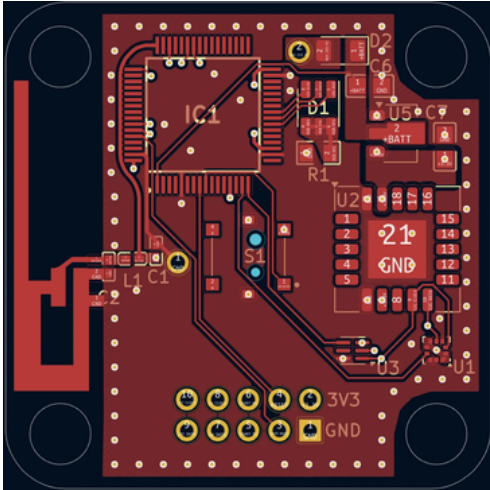
HOW?

- Determined chassis dimensions and materials by referencing an existing go-kart design.
- MIG-welded the frame using electrical conduit and ensured structural stability.
- Installed a Predator 224 gas engine, rear axle, wheels, steering, and suspension components.
- Converted the system to electric by removing the engine and integrating battery power, wiring, and drivetrain mounting.

RESULTS

- Achieved reliable starting, acceleration, and steering across test runs.
- Electric conversion produced slightly higher acceleration with reduced top speed.
- Gained hands-on experience in welding, mechanical assembly, electrical wiring, and troubleshooting under real-world constraints.
- Strengthened teamwork, patience, and iterative problem-solving skills through collaborative development.

ENVIRONMENTAL SENSOR BOARD - KICAD



WHAT?

- Designed a Bluetooth Low Energy environmental sensing monitor in a group of [three](#) people.
- Reads environmental data using: [SCD40](#) (temperature, humidity), [LPS22HB](#) (atmospheric pressure), and [APDS-9306](#) (ambient light) sensors.

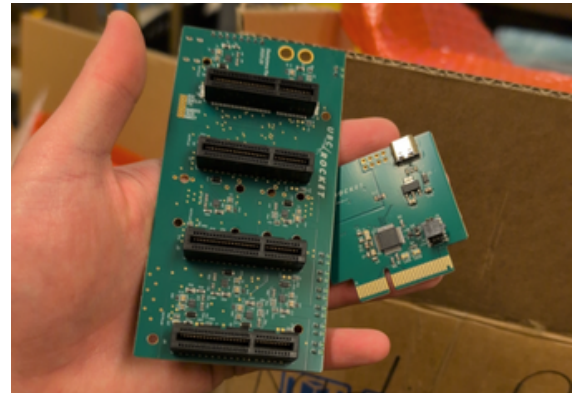
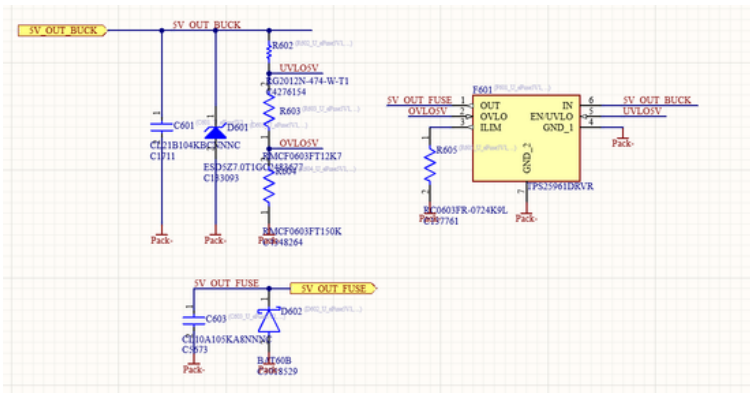
HOW?

- Created a 4-layer circuit board using [KiCad](#), including power filtering and debug connections. Programmed it to collect sensor data and send it wirelessly over Bluetooth.

RESULTS

- Successfully fabricated and assembled the board.
- The system was able to track temperature and humidity. Bluetooth and battery performance worked as expected.
- Gained hands-on experience with [RF layout](#), multilayer design, and [embedded programming](#).

ROCKET BACKPLANE - ALTIUM & GIT/GITHUB



WHAT?

- Designed an avionics [backplane PCB](#) for UBC Rocket that acts like a "hub board," letting multiple flight electronics boards plug in and share power and key signals.
- Collaborated with teammates using [Git](#) and [GitHub](#).

HOW?

- Translated the team's needs into a clear board plan (power rails, connector pinout, where signals go), then built the schematic and PCB layout in [Altium](#), including sensible protection parts ([e-fuse](#), surge protection, filtering).
- Read through datasheets to choose and apply an [e-fuse](#) solution for the backplane (power limit + protection behavior), then turned those specs into a working [Altium schematic](#).

RESULTS

- Made the avionics setup cleaner and more reliable by cutting down on [loose wiring](#), adding smarter power protection, and making the board easier to test and [debug](#) during integration.
- Built-in power limiting helped prevent "oops" moments ([shorts/miswires](#)) from taking down the whole avionics stack. The clearer schematic and test-friendly layout also made [troubleshooting](#) faster during bring-up and revisions.